Integritet baze podataka

> Materijal za vježbu i samostalno učenje iz SQL-a.
> Primjeri su pisani u stilu **SQL Servera** (`IDENTITY`, `NVARCHAR`, `CHECK`, `ALTER TABLE`).

Kako koristiti ovaj materijal

Preporučeni način rada:

1. pročitaj kratko objašnjenje
2. pokreni SQL primjer
3. usporedi rezultat s očekivanim ponašanjem
4. isprobaj izmjene samostalno
5. odgovori na pitanje za provjeru razumijevanja

> Napomena: Primjeri su prilagođeni SQL Serveru. U drugim sustavima baza podataka neki detalji mogu se razlikovati.

Ciljevi lekcije

Nakon ove lekcije student treba moći:

- objasniti pojam integriteta baze podataka
- razlikovati vrste integriteta
- koristiti `NOT NULL`, `DEFAULT`, `PRIMARY KEY`, `UNIQUE`, `CHECK` i `FOREIGN KEY`
- razumjeti razliku između `DELETE`, `TRUNCATE` i `DROP`
- definirati jednostavne i složene ključeve
- naknadno mijenjati definiciju tablice naredbom `ALTER TABLE`

---

1. Pojam integriteta baze podataka

Ideja

Integritet baze podataka odnosi se na konzistentnost i ispravnost podataka sadržanih u bazi podataka.

Integritet baze podataka može biti narušen zbog:

- slučajne pogreške korisnika pri unosu ili izmjeni podataka
- pogreške aplikacijskog programa ili sustava

Vrste integriteta

Promatrat ćemo sljedeće vrste integriteta:

- entitetski integritet (*Entity integrity*)
- integritet ključa (*Key integrity*)

- domenski integritet (*Domain integrity*)
- ograničenja `NULL` vrijednosti
- referencijski integritet (*Referential integrity*)
- opća integritetska ograničenja (*General integrity constraints*)

Zašto je to važno?

Ako baza ne bi imala integritetska ograničenja, bilo bi moguće:

- spremiti nepotpune podatke
- spremiti kontradiktorne podatke
- povezati zapis s nepostojećim zapisom u drugoj tablici
- unijeti više redaka koji bi trebali biti jedinstveni

Pitanje za razmišljanje

Zašto nije dovoljno da aplikacija provjerava ispravnost podataka, nego to mora raditi i sama baza?

---

2. Ograničenja `NULL` vrijednosti

Ideja

Ograničenje `NOT NULL` postiže se navođenjem rezerviranih riječi `NOT NULL` iza tipa podatka pri definiciji atributa.

To znači da atribut **mora imati vrijednost** i ne smije ostati nepoznat (`NULL`).

Primjer definicije tablice

```
CREATE TABLE nastavnik (
    sifNast INT NOT NULL,
    oibNast CHAR(11) NOT NULL,
    prezNast NVARCHAR(40)
);
```

Unos testnih podataka

```
INSERT INTO nastavnik VALUES
(1111, '06382780091', N'Pascal'),
(3333, '91643023865', N'Newton'),
(2222, '51843144239', NULL);

SELECT * FROM nastavnik;
```

Definicija tablice dozvoljava u stupcu prezNast unos NULL 'vrijednosti'.

Unos `NULL` vrijednosti u stupcu gdje je definirano `NOT NULL` ograničenje

```sql
INSERT INTO nastavnik VALUES (4444, NULL, N'Gauss');
```

Očekivanje: SQL Server vraća poruku tipa *Cannot insert the value NULL into column 'oibNast'...*

Pokušaj ponovnog unosa istog zapisa

```sql
INSERT INTO nastavnik VALUES (1111, '06382780091', N'Pascal');

SELECT * FROM nastavnik;
```

Što ovdje treba primijetiti?

Iako su `sifNast` i `oibNast` označeni kao `NOT NULL`, to **ne znači** da su automatski jedinstveni.

Drugim riječima, `NOT NULL` osigurava samo da vrijednost postoji, ali **ne sprječava duplikate**.

Brisanje podataka i tablice

```sql
DELETE FROM nastavnik;
```

```sql
TRUNCATE TABLE nastavnik;
```

```sql
DROP TABLE nastavnik;
```

Objašnjenje

- `DELETE` bez `WHERE` uvjeta također briše sve retke iz tablice
- `TRUNCATE` će isprazniti tablicu, odnosno obrisati sve retke
- `DROP` briše cijelu tablicu, odnosno i strukturu i podatke

Važna razlika između `DELETE` i `TRUNCATE`:

- `DELETE` bilježi svaki obrisani red u transaction logu, što omogućuje rollback
- `TRUNCATE` koristi minimalno logiranje i zato je brži

Praktično pravilo

- `DELETE` najčešće koristimo kada brišemo određene retke, često uz `WHERE`
- `TRUNCATE` koristimo kada želimo brzo isprazniti cijelu tablicu
- `DROP` koristimo kada nam tablica više ne treba

Pitanja za provjeru razumijevanja

1. Sprječava li `NOT NULL` unos duplikata?
2. Je li `NULL` isto što i prazan string?
3. Kada koristiš `DROP`, brišu li se samo podaci ili i struktura tablice?

DEFAULT

Ideja

- `NOT NULL` → atribut mora imati vrijednost
- `DEFAULT` → baza automatski dodjeljuje vrijednost ako nije unesena

---

Primjer

```
CREATE TABLE nastavnik (
    sifNast INT NOT NULL,
    oibNast CHAR(11) NOT NULL,
    prezNast NVARCHAR(40) NOT NULL,
    aktivan BIT DEFAULT 1
);
```

Unos bez navođenja stupca `aktivan`

```
INSERT INTO nastavnik (sifNast, oibNast, prezNast)
VALUES (1001, '06382780091', N'Newton');

SELECT * FROM nastavnik;
```

rezultat: `aktivan = 1` (postavljeno pomoću DEFAULT)

Unos s eksplicitnom vrijednošću

```
INSERT INTO nastavnik (sifNast, oibNast, prezNast, aktivan)
VALUES (1002, '91643023865', N'Maxwell', 0);

SELECT * FROM nastavnik;
```

Napomena:

`DEFAULT` definira početnu vrijednost pri unosu, ali tek uz `NOT NULL` postižemo potpuni integritet podataka jer onemogućujemo eksplicitni unos nevažećih NULL vrijednosti.

Npr. Ako definiramo stupac samo kao **aktivan** `BIT DEFAULT 1`, gubimo potpunu kontrolu nad integritetom. Iako `DEFAULT` sugerira da će nastavnik biti aktivan, baza će dopustiti sljedeće:

```
INSERT INTO nastavnik (sifNast, oibNast, prezNast, aktivan)
VALUES (1001, '06382780091', N'Newton', NULL);

SELECT * FROM nastavnik;
```

Dakle, potrebno je stupac **aktivan** definirati kao `BIT NOT NULL DEFAULT 1`

---

3. Primarni ključ (`PRIMARY KEY`)

Teorija

- **Entitetski integritet**: niti jedan atribut primarnog ključa ne smije poprimiti `NULL` vrijednost.
- **Integritet ključa**: u relaciji ne smiju postojati dva retka s jednakim vrijednostima ključa.

> U nastavku ćemo uglavnom koristiti riječ **redak**, iako se u teoriji relacijskih baza često koristi pojam **n-torka**.

Primjer definicije

```
DROP TABLE nastavnik;

CREATE TABLE nastavnik (
    sifNast INT PRIMARY KEY,
    oibNast CHAR(11) NOT NULL,
    prezNast NVARCHAR(40)
);
```

Kada se integritetsko ograničenje odnosi na samo jedan atribut, može se definirati neposredno uz definiciju atributa.

Unos podataka

```
INSERT INTO nastavnik VALUES
(1111, '06382780091', N'Pascal'),
(3333, '91643023865', N'Newton'),
(2222, '51843144239', N'Cantor');

SELECT * FROM nastavnik;
```

> Entitetski integritet i integritet ključa za primarni ključ uvijek se osiguravaju pomoću `PRIMARY KEY`.

☑ Ispravno ponašanje

Svaki redak ima različit `sifNast`, pa unos prolazi.

❌ Što se događa kod duplikata?

Pokušajmo ubaciti redak s ključem koji već postoji:

```
INSERT INTO nastavnik VALUES (1111, '06382780091', N'Pascal');
```

Očekivanje: SQL Server vraća poruku tipa *Violation of PRIMARY KEY constraint...*

Važna napomena

`PRIMARY KEY` automatski znači:

- vrijednost mora postojati (`NOT NULL`)
- vrijednost mora biti jedinstvena

Zato je `PRIMARY KEY` više od običnog `NOT NULL`.

Pitanja za provjeru razumijevanja

1. Zašto `PRIMARY KEY` ne dopušta `NULL`?
2. Može li tablica imati dva primarna ključa?
3. Koja je razlika između `NOT NULL` i `PRIMARY KEY`?

---

4. `UNIQUE` ograničenje

Ideja

`UNIQUE` se koristi za zaštitu **alternativnog ključa**.

Primarni ključ je glavni identifikator retka, a alternativni ključ je drugi atribut ili skup atributa koji također mora biti jedinstven.

Primjer: tablica `nastavnik`

```
DROP TABLE nastavnik;

CREATE TABLE nastavnik (
    sifNast INT PRIMARY KEY,
    oibNast CHAR(11) UNIQUE,
    prezNast NVARCHAR(40)
);
```

> Svaki alternativni ključ treba zaštititi `UNIQUE` ograničenjem.

Primjeri unosa

```
INSERT INTO nastavnik VALUES (1000, '06382780091', N'Newton');
INSERT INTO nastavnik VALUES (1000, '63743552278', N'Leibniz'); -- ne prolazi zbog
PRIMARY KEY
INSERT INTO nastavnik VALUES (1001, '06382780091', N'Codd'); -- ne prolazi zbog
UNIQUE
SELECT * FROM nastavnik;
```

Što ovdje treba primijetiti?

- prvi `INSERT` prolazi
- drugi ne prolazi jer je dupliciran `sifNast`
- treći ne prolazi jer je dupliciran `oibNast`

Dakle:

- `PRIMARY KEY` štiti primarni identifikator
- `UNIQUE` štiti alternativni identifikator

Važna napomena o `UNIQUE` i `NULL`

UNIQUE ograničenje u SQL Serveru ne dozvoljava pojavu više NULL vrijednosti. To se razlikuje od nekih drugih sustava, npr. Oraclea ili PostgreSQL-a.

```
INSERT INTO nastavnik VALUES (1006, NULL, N'Moore');
SELECT * FROM nastavnik;
```

```
INSERT INTO nastavnik VALUES (1007, NULL, N'Kepler'); --ne prolazi jer SQL server
ne dozvoljava pojavu više NULL vrijednosti
SELECT * FROM nastavnik;
```

Očekivanje: SQL Server vraća poruku tipa *Violation of UNIQUE KEY constraint...*

Pitanja za provjeru razumijevanja

1. Čemu služi `UNIQUE`?
2. Po čemu se `UNIQUE` razlikuje od `PRIMARY KEY`?
3. Kako se SQL Server ponaša kada `UNIQUE` stupac sadrži više `NULL` vrijednosti?

---

5. `UNIQUE` + `NOT NULL`

Ideja

Ako želimo osigurati da OIB:

- uvijek postoji

- i da je jedinstven

tada kombiniramo `NOT NULL` i `UNIQUE`.

Primjer

```sql
DROP TABLE nastavnik;

CREATE TABLE nastavnik (
    sifNast INT PRIMARY KEY,
    oibNast CHAR(11) NOT NULL UNIQUE,
    prezNast NVARCHAR(40)
);

INSERT INTO nastavnik VALUES (1001, '06382780091', N'Newton');
SELECT * FROM nastavnik;

INSERT INTO nastavnik VALUES (1005, NULL, N'Gauss');
```

Što očekivati?

Drugi `INSERT` ne prolazi jer je `oibNast` definiran kao `NOT NULL`, pa vrijednost mora postojati.

Pitanje za provjeru razumijevanja

Zašto drugi `INSERT` više ne prolazi, iako je u prethodnom primjeru bilo moguće imati jedan `NULL`?

---

6. Automatsko generiranje ključa pomoću `IDENTITY`

Ideja

Ponekad ne želimo ručno upisivati primarni ključ, nego želimo da ga baza generira automatski.

Definicija tablice

```sql
DROP TABLE nastavnik;

CREATE TABLE nastavnik (
    sifNast INT IDENTITY(5001, 1) PRIMARY KEY,
    oibNast CHAR(11) NOT NULL UNIQUE,
    prezNast NVARCHAR(40)
);
```

Objašnjenje

`IDENTITY` je svojstvo stupca koje automatski generira jedinstvene numeričke vrijednosti za svaki novi redak u tablici.

Primjeri:

- `IDENTITY(1, 1)` → prvi zapis kreće od 1, svaki sljedeći raste za 1
- `IDENTITY(100, 10)` → prvi zapis kreće od 100, svaki sljedeći raste za 10
- `IDENTITY(5001, 1)` → prvi zapis kreće od 5001, svaki sljedeći raste za 1

Unos podataka

> Kod unosa podataka ne navodimo **sifNast** jer se on generira automatski

```
INSERT INTO nastavnik VALUES
('06382780091', N'Newton'),
('91643023865', N'Maxwell');

SELECT * FROM nastavnik;
```

Razlika između `DELETE` i `TRUNCATE`

DELETE briše redove iz tablice, ali ne resetira IDENTITY vrijednost.

```
DELETE FROM nastavnik;

INSERT INTO nastavnik VALUES ('06382780091', N'Newton');

SELECT * FROM nastavnik;
```

TRUNCATE briše sve redove iz tablice i resetira IDENTITY brojač na početnu vrijednost.

```
TRUNCATE TABLE nastavnik;

INSERT INTO nastavnik VALUES ('06382780091', N'Newton');

SELECT * FROM nastavnik;
```

Pitanja za provjeru razumijevanja

1. Zašto kod `IDENTITY` više ne navodimo vrijednost za `sifNast`?
2. Što resetira `IDENTITY` brojač: `DELETE` ili `TRUNCATE`?
3. Koja je prednost surogatnog ključa?

---

7. Složeni primarni ključ

Ideja

Do sada su integritetska ograničenja bila definirana uz pojedini atribut.
To je dobro kada se ključ sastoji od jednog stupca.

Ako se ključ sastoji od više stupaca, tada govorimo o **složenom ključu**.

Primjer koji nije ispravan

```
CREATE TABLE ispit (
    jmbag CHAR(10) PRIMARY KEY,
    sifPred INT PRIMARY KEY,
    datIspit DATE PRIMARY KEY,
    ocj TINYINT NOT NULL,
    sifNast INT
);
```

> Na ovaj način ne može se definirati složeni ključ.

Zašto je ovo pogrešno?

Tablica može imati samo **jedan** primarni ključ.
Ako ključ čini više stupaca, oni se moraju navesti zajedno u jednoj definiciji ograničenja.

Ispravno rješenje

```
CREATE TABLE ispit (
    jmbag CHAR(10),
    sifPred INT,
    datIspit DATE,
    ocj TINYINT NOT NULL,
    sifNast INT,
    CONSTRAINT PK_ispit PRIMARY KEY (jmbag, sifPred, datIspit)
);
```

Što je table-level ograničenje?

`Table-level` ograničenje definira se nakon svih stupaca, u posebnom dijelu `CREATE TABLE` izraza.

Prednosti:

- omogućuje imenovanje ograničenja pomoću `CONSTRAINT ime`
- obvezno je kod višestupčanih ograničenja
- korisno je za čitljivost kod složenijih pravila

Primjeri unosa

```
INSERT INTO ispit VALUES
('0555004388', 1001, '2022-01-29', 1, 1111),
```

```
('0555004388', 1001, '2022-02-05', 3, 1111),
('0555004388', 1003, '2021-06-28', 2, 3333),
('0555004388', 1002, '2021-06-27', 2, 2222),
('2902984555', 1001, '2022-01-29', 3, 2222);

SELECT * FROM ispit;
```

Pokušajmo unijeti vrijednost ključa koji postoji:

```
INSERT INTO ispit VALUES ('0555004388', 1001, '2022-01-29', 1, 1111);
```

Što očekivati?

SQL Server vraća grešku: *Violation of **PRIMARY KEY** constraint 'PK_ispit'. Cannot insert duplicate key in object 'dbo.ispit'. The duplicate key value is (0555004388, 1001, 2022-01-29).*

Surogatni ključ + zaštita alternativnog ključa

Kada imamo složeni ključ, često uvodimo **surogatni ključ**, a prirodni ključ štitimo pomoću `UNIQUE`.

```
DROP TABLE ispit;

CREATE TABLE ispit
(
    sifIspit INT IDENTITY(1000,1) PRIMARY KEY, -- surogatni samonumerirajući ključ
    jmbag CHAR(10) NOT NULL,
    sifPred INT NOT NULL,
    datIspit DATE NOT NULL,
    ocj TINYINT NOT NULL,
    sifNast INT,
    CONSTRAINT UQ_ispit UNIQUE (jmbag, sifPred, datIspit) -- štitimo alternativni
ključ
);

-- Testni podaci
INSERT INTO ispit VALUES
('0555004388', 1001, '2022-01-29', 1, 1111),
('0555004388', 1001, '2022-02-05', 3, 1111),
('0555004388', 1003, '2021-06-28', 2, 3333),
('0555004388', 1002, '2021-06-27', 2, 2222),
('2902984555', 1001, '2022-01-29', 3, 2222);

SELECT * FROM ispit;
```

Zašto je to korisno?

- `sifIspit` je jednostavan za referenciranje

- prirodna kombinacija (`jmbag`, `sifPred`, `datIspit`) i dalje ostaje zaštićena od duplikata

Pokušajmo unijeti vrijednost ključa koji postoji:

```
INSERT INTO ispit VALUES ('0555004388', 1001, '2022-01-29', 1, 1111);
```

Što očekivati?

SQL Server vraća grešku: *Violation of **UNIQUE KEY** constraint 'UQ_ispit'. Cannot insert duplicate key in object 'dbo.ispit'. The duplicate key value is (0555004388, 1001, 2022-01-29).*

Pitanje za razmišljanje

Što bi se moglo dogoditi kada ne bismo zaštitili alternativni ključ pomoću `UNIQUE`?

---

8. `CHECK` ograničenje

Ideja

Domenski integritet djelomično je osiguran već samim tipom podataka.

Primjer:

- `SMALLINT` dopušta cijele brojeve u intervalu od `-32768` do `32767`

Ponekad želimo dodatno suziti dopuštene vrijednosti.
Tada koristimo `CHECK`.

Primjer

```
DROP TABLE ispit;

CREATE TABLE ispit
(
    sifIspit INT IDENTITY(1000,1) PRIMARY KEY,
    jmbag CHAR(10) NOT NULL,
    sifPred INT NOT NULL,
    datIspit DATE NOT NULL,
    ocj TINYINT CHECK (ocj BETWEEN 1 AND 5) NOT NULL,
    sifNast INT,
    CONSTRAINT UQ_ispit UNIQUE (jmbag, sifPred, datIspit)
);
```

Ovdje se osigurava da je ocjena uvijek između 1 i 5.

Provjera

```sql
INSERT INTO ispit VALUES
('0555004388', 1001, '2022-01-29', 1, 1111),
('2902984555', 1001, '2022-01-29', 6, 2222); --primijetite da je se unosi ocjena 6

SELECT * FROM ispit;
```

Što očekivati?

SQL Server vraća grešku tipa *The INSERT statement conflicted with the CHECK constraint...*

Važna napomena

Ako u jednoj `INSERT` naredbi pokušamo ubaciti više redaka, a jedan od njih krši `CHECK`, cijela naredba može biti odbijena.

Pitanja za provjeru razumijevanja

1. Zašto `CHECK` spada u domenski integritet?
2. Koje bi još poslovno pravilo moglo biti zapisano pomoću `CHECK`?

---

9. Naknadne izmjene definicije tablice (`ALTER TABLE`)

Ideja

Tablicu nije nužno u potpunosti definirati pri prvom stvaranju.
Kasnije joj možemo dodavati stupce, mijenjati tipove i postavljati nova ograničenja.

Polazna tablica

```sql
DROP TABLE nastavnik;

CREATE TABLE nastavnik (
    sifNast INT IDENTITY(5001, 1) PRIMARY KEY,
    oibNast CHAR(11) NOT NULL UNIQUE,
    prezNast NVARCHAR(10)
);

INSERT INTO nastavnik VALUES
('06382780091', N'Newton'),
('91643023865', N'Maxwell');

SELECT * FROM nastavnik;
```

Promjena tipa stupca

Hoće li proći sljedeći unos?

```
INSERT INTO nastavnik VALUES ('75114131579', N'Mohorovičić');
```

SQL Server vraća grešku tipa: *String or binary data would be truncated in table 'dbo.nastavnik', column 'prezNast'. Truncated value: 'Mohoroviči'*. Dakle, pokušavamo upisati 11 znakova u vrijednost atributa **prezNast** koji je definiran kao `NVARCHAR(10)`

Kako promijeniti definiciju?

```
ALTER TABLE nastavnik
ALTER COLUMN prezNast NVARCHAR(20);
```

Novi pokušaj:

```
INSERT INTO nastavnik VALUES ('75114131579', N'Mohorovičić');

SELECT * FROM nastavnik;
```

Dodavanje `NOT NULL` ograničenja

```
ALTER TABLE nastavnik
ALTER COLUMN prezNast NVARCHAR(20) NOT NULL;
```

Važna napomena

Prije postavljanja `NOT NULL`, svi postojeći retci moraju imati vrijednost u tom stupcu.

> Pravilo: novo ograničenje može se dodati ili pooštriti samo ako ga svi postojeći podaci već zadovoljavaju.

Dodavanje stupca

```
ALTER TABLE nastavnik
ADD imeNast NVARCHAR(10);

select * from nastavnik;
```

Unos podataka

```
INSERT INTO nastavnik VALUES ('60440865410', N'Einstein', 'Albert');

SELECT * FROM nastavnik;
```

Brisanje stupca

```
ALTER TABLE nastavnik
DROP COLUMN imeNast;

select * from nastavnik;
```

Pitanja za provjeru razumijevanja

1. Što se može mijenjati naredbom `ALTER TABLE`?
2. Zašto `ALTER COLUMN ... NOT NULL` nekad ne prolazi?
3. Mora li baza provjeriti postojeće podatke prije promjene?

---

10. Naknadno dodavanje ograničenja

Ideja

Ograničenja ne moramo uvijek definirati odmah prilikom `CREATE TABLE`.
Možemo ih dodati i naknadno, ali samo ako postojeći podaci već zadovoljavaju novo pravilo.

Nova verzija tablice

```
DROP TABLE nastavnik;

CREATE TABLE nastavnik (
    sifNast INT,
    oibNast CHAR(11) NOT NULL,
    prezNast NVARCHAR(40)
);

INSERT INTO nastavnik VALUES (1000, '06382780091', N'Newton');
INSERT INTO nastavnik VALUES (1000, '91643023865', N'Maxwell'); --unosimo dva puta
istu vrijednost za sifNast

select * from nastavnik;
```

Dodavanje `UNIQUE` ograničenja

```
ALTER TABLE nastavnik
ADD CONSTRAINT UQ_nastavnik_oib UNIQUE (oibNast);
```

Objašnjenje

SQL Server pokušava postaviti `UNIQUE` ograničenje nad stupcem `oibNast`.
Budući da su OIB-ovi različiti, naredba prolazi uspješno.

> Ograničenje se može dodati samo ako svi postojeći podaci već poštuju to pravilo.

Dodavanje primarnog ključa

```
ALTER TABLE nastavnik
ADD CONSTRAINT PK_nastavnik PRIMARY KEY (sifNast);
```

SQL Server vraća grešku: *Cannot define PRIMARY KEY constraint on **nullable column** in table 'nastavnik'.*

Što se događa?

Ovdje dobivamo problem jer:

- `PRIMARY KEY` ne dopušta `NULL`
- `PRIMARY KEY` ne dopušta duplikate

U našim podacima `sifNast` ima duplikat vrijednosti `1000`, pa dodavanje ključa ne prolazi.

Ispravljanje problema

Prvo osiguravamo da stupac ne dopušta `NULL`:

```
ALTER TABLE nastavnik
ALTER COLUMN sifNast INT NOT NULL;
```

Novi pokušaj dodavanja primarnog ključa:

```
ALTER TABLE nastavnik
ADD CONSTRAINT PK_nastavnik PRIMARY KEY (sifNast);
```

Sada SQL Server vraća grešku: *The CREATE UNIQUE INDEX statement terminated because a **duplicate key** was found for the object name 'dbo.nastavnik' and the index name 'PK_nastavnik'. The duplicate key value is (1000).*

Uklonimo duplikat:

```
--nastavniku s OIB-om 91643023865 mijenjamo šifru iz 1000 u 1001
UPDATE nastavnik
SET sifNast = 1001
WHERE oibNast = '91643023865';

--sada više nema duplih vrijednosti
select * from nastavnik;
```

Sada možemo ponovno pokušati:

```
ALTER TABLE nastavnik
ADD CONSTRAINT PK_nastavnik PRIMARY KEY (sifNast);
```

Pitanja za provjeru razumijevanja

1. Zašto dodavanje `UNIQUE` ograničenja može proći, a `PRIMARY KEY` ne?
2. Koja dva uvjeta stupac mora zadovoljiti da bi mogao postati `PRIMARY KEY`?
3. Zašto baza provjerava stare podatke prije dodavanja novog ograničenja?

---

11. Strani ključ (`FOREIGN KEY`)

Ideja

Referencijski integritet znači da se redak iz jedne tablice može pozivati na redak iz druge tablice samo ako taj zapis stvarno postoji.

Drugim riječima:

- ne možemo upisati povezan zapis ako roditeljski zapis ne postoji
- ne možemo obrisati roditeljski zapis ako na njega postoje povezni zapisi, osim ako nije definirana posebna akcija

Priprema tablica

Kreirajmo tablice `nastavnik`, `student` i `predmet`:

```
DROP TABLE nastavnik;

CREATE TABLE nastavnik(
    sifNast INT PRIMARY KEY,
    oibNast CHAR(11) NOT NULL UNIQUE,
    prezNast NVARCHAR(40)
);

CREATE TABLE student(
    jmbag CHAR(10) PRIMARY KEY,
    prezime NVARCHAR(20),
    ime NVARCHAR(20)
);

CREATE TABLE predmet(
    sifPred INT PRIMARY KEY,
    nazPred NVARCHAR(20)
);
```

Kreiranje tablice `ispit`

```
DROP TABLE ispit;

CREATE TABLE ispit
(
    sifIspit INT IDENTITY(1000,1) PRIMARY KEY,
    jmbag CHAR(10) NOT NULL,
    sifPred INT NOT NULL,
    datIspit DATE NOT NULL,
    ocj TINYINT NOT NULL CHECK (ocj BETWEEN 1 AND 5),
    sifNast INT,
    CONSTRAINT UQ_ispit UNIQUE (jmbag, sifPred, datIspit),
    CONSTRAINT FK_ispit_stud FOREIGN KEY (jmbag) REFERENCES student(jmbag),
    CONSTRAINT FK_ispit_predmet FOREIGN KEY (sifPred) REFERENCES predmet(sifPred),
    CONSTRAINT FK_ispit_nastavnik FOREIGN KEY (sifNast) REFERENCES
nastavnik(sifNast)
);
```

Ubacivanje testnih podataka

```
INSERT INTO nastavnik VALUES
(1111, '06382780091', N'Pascal'),
(3333, '91643023865', N'Newton'),
(2222, '51843144239', N'Cantor');

INSERT INTO student VALUES
('0555004388', N'Horvat', N'Ivan'),
('2902984555', N'Kolar', N'Petar');

INSERT INTO predmet VALUES
(1001, N'Mat-1'),
(1002, N'Mat-2'),
(1003, N'Fiz-1');

INSERT INTO ispit VALUES
('0555004388', 1001, '2022-01-29', 1, 1111),
('0555004388', 1001, '2022-02-05', 3, 1111),
('0555004388', 1003, '2021-06-28', 2, 3333),
('0555004388', 1002, '2021-06-27', 2, 2222),
('2902984555', 1001, '2022-01-29', 3, 2222);
```

❌ Pokušaj brisanja roditeljskog zapisa

```
--Provjera trenutnog stanja u tablicama student i ispit
SELECT * FROM student;
SELECT * FROM ispit;
```

```sql
--Pokušaj brisanja studenta s JMBAG-om 0555004388
DELETE FROM student WHERE jmbag = '0555004388';
```

Očekivanje: SQL Server odbija brisanje uz poruku tipa
The DELETE statement conflicted with the REFERENCE constraint "FK_ispit_stud"...

Zašto?

Zato što u tablici `ispit` postoje retci koji se pozivaju na postojeće studente.
Kad bi baza dozvolila brisanje, ostali bi "viseći" zapisi bez valjanog roditelja.

❌ Pokušaj unosa ispita za nepostojeći predmet

```sql
INSERT INTO ispit VALUES
('0555004388', 9999, '2024-01-20', 1, 1111);
```

Očekivanje: SQL Server odbija upis s greškom tipa
The INSERT statement conflicted with the FOREIGN KEY constraint "FK_ispit_predmet"...

Važna napomena

Kroz `FOREIGN KEY` osigurava se da se ne može unijeti zapis koji se poziva na nepostojeći zapis u povezanoj tablici.

Pitanja za provjeru razumijevanja

1. Zašto ne možemo obrisati studenta koji već ima evidentirane ispite?
2. Zašto ne možemo unijeti ispit za predmet koji ne postoji?
3. Koja je uloga `FOREIGN KEY` ograničenja?

---

12. Bonus: Kompenzacijske akcije

Ideja

Kod stranih ključeva moguće je definirati što će se dogoditi ako pokušamo obrisati roditeljski zapis.

Podržane opcije

- `ON DELETE NO ACTION` — zadana opcija; DBMS odbija operaciju brisanja roditeljskog zapisa
- `ON DELETE SET NULL` — vrijednosti stranog ključa postavlja na `NULL`
- `ON DELETE SET DEFAULT` — vrijednosti stranog ključa postavlja na zadanu vrijednost
- `ON DELETE CASCADE` — briše i povezane retke u zavisnoj tablici

> Oprez: `ON DELETE CASCADE` može obrisati velik broj povezanih redaka. Koristi se samo kada je takvo ponašanje stvarno poslovno pravilo.

Primjer: dodajmo `ON DELETE CASCADE`

```sql
DROP TABLE ispit;

CREATE TABLE ispit
(
    sifIspit INT IDENTITY(1000,1) PRIMARY KEY,
    jmbag CHAR(10) NOT NULL,
    sifPred INT NOT NULL,
    datIspit DATE NOT NULL,
    ocj TINYINT NOT NULL CHECK (ocj BETWEEN 1 AND 5),
    sifNast INT,
    CONSTRAINT UQ_ispit UNIQUE (jmbag, sifPred, datIspit),
    CONSTRAINT FK_ispit_stud FOREIGN KEY (jmbag) REFERENCES student(jmbag) ON
DELETE CASCADE,
    CONSTRAINT FK_ispit_predmet FOREIGN KEY (sifPred) REFERENCES predmet(sifPred),
    CONSTRAINT FK_ispit_nastavnik FOREIGN KEY (sifNast) REFERENCES
nastavnik(sifNast)
);

--testni podaci
INSERT INTO ispit VALUES
('0555004388', 1001, '2022-01-29', 1, 1111),
('0555004388', 1001, '2022-02-05', 3, 1111),
('0555004388', 1003, '2021-06-28', 2, 3333),
('0555004388', 1002, '2021-06-27', 2, 2222),
('2902984555', 1001, '2022-01-29', 3, 2222);
```

Provjera ponašanja

```sql
-- provjera prije brisanja
SELECT * FROM student;
SELECT * FROM ispit;

--Pokušaj (ovaj put uspješan!) brisanja studenta s JMBAG-om 0555004388
DELETE
FROM student
WHERE jmbag = '0555004388';

-- provjera nakon brisanja
SELECT * FROM student;
SELECT * FROM ispit;
```

Što očekivati?

Nakon brisanja studenta s `jmbag = '0555004388'`, automatski se brišu i svi njegovi povezani retci u tablici `ispit`.

Pitanja za provjeru razumijevanja

1. Koja je razlika između `NO ACTION` i `CASCADE`?
2. Kada bi `ON DELETE CASCADE` bio koristan?
3. Zašto njegova upotreba može biti opasna?

Sažetak ograničenja

Ograničenje	Svrha	Dopušta `NULL`	Dopušta duplikate
`NOT NULL`	zahtijeva obaveznu vrijednost	ne	da
`PRIMARY KEY`	jedinstveno identificira redak	ne	ne
`UNIQUE`	osigurava jedinstvenost alternativnog ključa	ovisi o sustavu	ne
`CHECK`	ograničava dopuštene vrijednosti	ovisi o definiciji	da
`FOREIGN KEY`	osigurava referencijski integritet	da, ako nije dodatno zabranjen	da

---

Završni sažetak

U ovoj lekciji naučili smo:

- `NOT NULL` sprječava nepoznate vrijednosti u obaveznim stupcima
- `DEFAULT` baza automatski dodjeljuje vrijednost ako nije unesena
- `PRIMARY KEY` osigurava jedinstvenu identifikaciju redaka
- `UNIQUE` štiti alternativne ključeve
- `CHECK` ograničava dopuštene vrijednosti u stupcu
- `FOREIGN KEY` osigurava referencijski integritet između tablica
- `ALTER TABLE` omogućuje naknadne promjene strukture i ograničenja